SmartStock AI Inventory Management System

Complete Project Analysis and Data Flow Diagrams

Prepared for Odoo Internship | 2026-10-28

Executive Summary

SmartStock AI is a Flask inventory application integrated with Odoo PostgreSQL tables. It supports authentication, products, stock, purchasing, sales, reports, exports, audit history, and reviews.

Technology Stack

Layer	Technology	Purpose
Backend	Python + Flask	Application server, routes, authentication, business logic
Database	PostgreSQL	Persistent storage for Odoo and SmartStock application data
ERP integration	Odoo PostgreSQL tables	Products, variants, stock quantities, and prices
Database driver	psycopg2-binary	Python-to-PostgreSQL connectivity
Frontend	HTML5, CSS3, JavaScript	Browser interface and API interaction
UI	Bootstrap 5 + Bootstrap Icons	Responsive layout, controls, and icons
Exports	ReportLab + openpyxl + CSV	PDF, Excel, and CSV reports

Core Modules

Module	Description
Authentication	Signup, login, logout, hashed passwords, sessions, reset-token flow
Authorization	Admin and user roles with protected admin APIs
Product Management	Create, read, update, delete, search, pagination, product details
Inventory	Stock-in/out, negative-stock prevention, transaction history
Purchasing and Sales	Purchase increases stock; sales decrease stock with validation
Master Data	Categories, suppliers, and customers
Reports and Exports	Summary metrics, charts, audit report, CSV, Excel, and PDF
Reviews	Product ratings, comments, average rating, and review listing

Project Files and APIs

File	Responsibility
app.py	Flask application factory, page routes, error handling, schema startup
routes.py	Authentication, product CRUD, stock, purchase, sales, reports, exports, reviews APIs
database.py	PostgreSQL connection management and application schema initialization
config.py	Database and Flask configuration through environment variables
requirements.txt	Python dependencies
index.html	Inventory dashboard
product-management.html	Dedicated product management screen
product-details.html / reviews.html	Product detail and review pages
purchase.html / sales.html	Purchase and sales transaction pages
static/*.js	Dashboard, authentication, management, advanced, and review interactions
static/style.css	Shared responsive styling

Area	Important endpoints
Auth	POST /api/auth/signup, POST /api/auth/login, POST /api/auth/logout, POST /api/auth/forgot-password
Products	GET/POST /api/products, PUT/DELETE /api/products/<id>
Inventory	POST /api/stock-transactions, GET /api/inventory-history
Business	POST/GET /api/purchases, POST/GET /api/sales
Master data	GET/POST /api/categories, /api/suppliers, /api/customers
Reports	GET /api/reports, /api/audit
Exports	GET /api/exports/products.csv, .xlsx, .pdf
Reviews	GET/POST /api/products/<id>/reviews

DFD - Context Level

The following diagram shows the main external entities, central application process, database, and outputs.

User / Administrator	<-->	SmartStock Flask Application	<-->	Odoo PostgreSQL + SmartStock Tables
		|		
		v		
		Dashboard / JSON / CSV / Excel / PDF Outputs		

DFD - Level 1 Processes

Process	Input	Data Store / Output
1. Authentication	Email, password, reset request	smartstock_users, reset tokens, session
2. Product Management	Product name, quantity, price	Odoo product_template, product_product, stock_quant
3. Inventory Transactions	Stock-in/out quantity	stock_quant, stock transactions, audit history
4. Purchase and Sales	Order product, quantity, price	Purchase/sales tables and stock quantities
5. Reports and Exports	Report request	Dashboard metrics, audit data, CSV/XLSX/PDF
6. Reviews	Rating and comment	Product reviews table and product detail response

Security, Validation and Production Notes

Passwords are hashed, APIs are session protected, admin APIs check roles, negative stock is prevented, ratings are restricted to 1-5, and user-provided text is escaped before rendering. Recommended production improvements include CSRF protection, rate limiting, email delivery for reset tokens, formal migrations, automated tests, and Odoo ORM/API usage.

Conclusion

The project is suitable for an Odoo internship demonstration because it connects Python application logic with PostgreSQL and Odoo inventory structures while implementing a complete inventory workflow.